

Examen Módulo 3

17/05/2024

Nombre: _____ Lab: _____ Horario: _____

Este enunciado debe ser devuelto al ayudante a cargo de la sala. Presentar credencial o cédula de identidad.

Instrucciones

El presente examen tiene tres partes. Cada parte indica su puntaje y el nombre del archivo que se debe entregar. Tendrás un tiempo de 110 minutos para resolver el examen.

Debes entregar a través de Canvas un archivo para cada parte con el nombre que se especifica en el enunciado. Para ver la salida esperada en cada una de las partes del examen, puedes revisar una lista de reproducción en YouTube (<https://bit.ly/20240517-ing1103-m3>) con las demostraciones de los programas

Sistema de Puntaje

Todos los ítemes son evaluados con una escala de 1-5 (1: no presenta solución, 2: presenta esbozo de solución incompleto o con errores graves, 3: solución incompleta o imprecisa, 4: error(es) mínimo(s), 5: excelente) y luego un ponderador (1:0, 2:0,25, 3:0,5, 4:0,75, 5:1,0) es aplicado al puntaje del ítem. El puntaje otorgado a la solución se calcula como la suma de los puntajes parciales obtenidos, aplicándose los ponderadores.

Parte 1 (2 puntos)

1. [5] La sucesión de Fibonacci se define de la siguiente manera:

$$F(0) = 0, \quad F(1) = 1$$

y para $n \geq 2$:

$$F(n) = F(n-1) + F(n-2)$$

Crea un archivo de código Python (**p1-1.py**) que implemente la función **fibonacci(n)**. La función retorna una lista con los primeros n términos de la sucesión. A continuación de la declaración de la función **fibonacci**, escribe código (al que nos referiremos como "código de prueba") que llame a la función con $n = 15$, reciba su resultado, sume los quince términos en la lista, y despliegue la lista junto con dicha suma. El resultado se tiene que ver de la siguiente forma:

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]  
986
```

2. [5] Crea una copia del programa anterior y llámala **p1-2.py**. Ahora, modifica la función **fibonacci(n, filas, columnas)**, de que la función continúe generando los valores de la sucesión de Fibonacci, pero retorne una lista conformada por **filas** listas anidadas, y que cada lista anidada vayan siendo llenada con **columnas** valores consecutivos de la sucesión. Si los n valores de la sucesión se terminan antes de llenar todas las filas y columnas necesarias, la función debe rellenar con ceros todas las listas anidadas hasta completar las **filas** requeridas. Ejemplos:

```
>>> fibonacci(4,2,2)
[[0, 1], [1, 2]]
>>> fibonacci(12,4,4)
[[0, 1, 1, 2], [3, 5, 8, 13], [21, 34, 55, 89], [0, 0, 0, 0]]
>>> fibonacci(16,4,4)
[[0, 1, 1, 2], [3, 5, 8, 13], [21, 34, 55, 89], [144, 233, 377, 610]]
```

Nota: No dediques tiempo excesivo a implementar esta función y que quede perfecta. Hay puntaje por logro parcial. Las partes siguientes serán evaluadas sin penalización si no logras el funcionamiento perfecto aquí.

3. [5] Una vez implementada la función `fibonacci(n, filas, columnas)` en `p1-2.py`, modifica el código de prueba en el programa para solicitar al usuario los valores `n`, `filas` y `columnas`. Luego, el código de prueba debe llamar a la función `fibonacci` pasando los parámetros requeridos, calcular y desplegar la sumatoria de los elementos en la lista de listas retornada por la función.
4. [5] Crea una última copia del programa y llámala `p1-3.py`. Agrega una función llamada `despliega_fibonacci(l)`, en donde `l` es una lista de listas con el formato que retorna la función `fibonacci`. La función `despliega_fibonacci(l)` debe desplegar la lista `l` por filas y columnas con el siguiente formato (ejemplo):

0	1	1	2
3	5	8	13
21	34	55	89
144	233	377	610

No es necesario que la función retorne un valor. Finalmente, modifica el código de prueba para que junto con desplegar la suma de los términos, llame a la función `despliega_fibonacci(l)` para mostrar la lista de listas completa. Es importante notar que `despliega_fibonacci(l)` debe desplegar listas de listas de cualquier tamaño. Para lograr esto es posible usar la función `len()`. Hint: `len(l)` y `len(l[0])`.

Solución

Listing 1: Solución de la Parte 1-1

```
1 def fibonacci(n):
2     l = [0,1]
3     for i in range(n-2):
4         l.append(l[-2:-1][0] + l[-1:][0])
5     return l
6
7 lf = fibonacci(15)
8 suma = 0
9 for i in range(len(lf)):
10     suma += lf[i]
11
12 print(lf)
13 print(suma)
```

Listing 2: Solución de la Parte 1-2

```
1 def fibonacci(n, filas, columnas):
2     l = [0,1]
3     m = []
4     count = 0
5     for i in range(filas):
6         if i == 0:
```

```
7     fila = [0,1]
8     count += 2
9     else:
10        fila = []
11    for j in range(columnas-len(fila)):
12        l.append(l[-2:-1][0] + l[-1:][0])
13        if count < n:
14            fila.append(l[-1])
15            count += 1
16        else:
17            fila.append(0)
18    m.append(fila)
19    return m
20
21 n = int(input("n>"))
22 filas = int(input("filas>"))
23 columnas = int(input("columnas>"))
24
25 lf = fibonacci(n, filas, columnas)
26 suma = 0
27 for i in range(len(lf)):
28     for j in range(len(lf[0])):
29         suma += lf[i][j]
30
31 print(lf)
32 print(suma)
```

Listing 3: Solución de la Parte 1-3

```
1 def fibonacci(n, filas, columnas):
2     l = [0,1]
3     m = []
4     count = 0
5     for i in range(filas):
6         if i == 0:
7             fila = [0,1]
8             count += 2
9         else:
10            fila = []
11        for j in range(columnas-len(fila)):
12            l.append(l[-2:-1][0] + l[-1:][0])
13            if count < n:
14                fila.append(l[-1])
15                count += 1
16            else:
17                fila.append(0)
18        m.append(fila)
19    return m
20
21 def despliega_fibonacci(l):
22     for i in range(len(l)):
23         for j in range(len(l[0])):
24             print(f"{l[i][j]:<9} ", end="")
25     print()
26
27 n = int(input("n>"))
28 filas = int(input("filas>"))
29 columnas = int(input("columnas>"))
30
31 lf = fibonacci(n, filas, columnas)
32
33 suma = 0
34 for i in range(len(lf)):
35     for j in range(len(lf[0])):
36         suma += lf[i][j]
37
38 print(suma)
39 despliega_fibonacci(lf)
```

Parte 2 (p2.py; 2 puntos)

Seguiremos extendiendo el programa desarrollado en la parte anterior (p1-3.py). Para esto, crea una copia y llámala (p2.py). A continuación, nos referiremos a la lista de listas generada por `fibonacci(n, filas, columnas)` como “matriz Fibonacci”.

1. [.75] Crea una función llamada `desordenar(l)`, que reciba una lista de listas `l` y desordene su contenido. Para esto, implementa un recorrido completo por `l`, tal que para cada elemento (escalar) en `l` la función genera al azar índices de fila y columnas válidos en `l` y luego realiza intercambio de los elementos en la posición actual del recorrido y en la posición aleatoria generada. Actualiza el código de prueba del programa para que quede de la siguiente forma:

```
# codigo de prueba:
m = fibonacci(n, fil, col) # n, fil y col deben ser pedidos al usuario antes
despliega_fibonacci(m)
print() # saltarse linea
desordenar(m)
despliega_fibonacci(m)
```

2. [.5] Crea una función llamada `estadisticas(l)`, en donde el parámetro `l` es una lista de listas. La función debe retornar una lista con tres elementos: promedio, valor mínimo y valor máximo en `l`.
3. [.75] Crea una función llamada `sub_fibonacci(l, fila, columna)`, que retorne una parte de la “matriz Fibonacci” (parámetro `l`) comenzando en la `fila` y la `columna` dadas como parámetros, hasta la última fila y columna de `l`. Notar que `fila` y `columna` parten en 1. Actualiza el código de prueba para que quede de la siguiente forma:

```
# codigo de prueba:
m = fibonacci(n, fil, col) # n, fil y col deben ser pedidos al usuario
despliega_fibonacci(m)
print()
desordenar(m)
despliega_fibonacci(m)
print()
sm = sub_fibonacci(m, 2, 2)
despliega_fibonacci(sm)
print(estadisticas(sm))
```

Solución

Listing 4: Solución de la Parte 2

```
1  from numpy import random
2
3  def fibonacci(n, filas, columnas):
4      l = [0,1]
5      m = []
6      count = 0
7      for i in range(filas):
8          if i == 0:
9              fila = [0,1]
10             count += 2
11         else:
12             fila = []
13             for j in range(columnas-len(fila)):
14                 l.append(l[-2:-1][0] + l[-1:] [0])
15                 if count < n:
16                     fila.append(l[-1])
17                     count += 1
18             else:
19                 fila.append(0)
20             m.append(fila)
21         return m
22
```

```
23 def despliega_fibonacci(l):
24     for i in range(len(l)):
25         for j in range(len(l[0])):
26             print(f"{l[i][j]:<9}", end=" ")
27         print()
28
29 def desordenar(l):
30     for i in range(len(l)):
31         for j in range(len(l[0])):
32             ir = random.randint(len(l))
33             jr = random.randint(len(l[0]))
34             aux = l[i][j]
35             l[i][j] = l[ir][jr]
36             l[ir][jr] = aux
37
38 def estadisticas(l):
39     suma = 0
40     n = len(l) * len(l[0])
41
42     mn = l[0][0]
43     mx = l[0][0]
44
45     for i in range(len(l)):
46         for j in range(len(l[0])):
47             suma += l[i][j]
48             if l[i][j] < mn:
49                 mn = l[i][j]
50             if l[i][j] > mx:
51                 mx = l[i][j]
52     return [suma/n, mn, mx]
53
54 def sub_fibonacci(l, fila, columna):
55     mfb = []
56     for i in range(fila-1, len(l)):
57         mfb.append(l[i][columna-1:])
58     return mfb
59
60 n = int(input("n>"))
61 filas = int(input("filas>"))
62 columnas = int(input("columnas>"))
63
64 lf = fibonacci(n, filas, columnas)
65
66 despliega_fibonacci(lf)
67 print()
68 desordenar(lf)
69 despliega_fibonacci(lf)
70 print()
71 sf = sub_fibonacci(lf, 2, 2)
72 despliega_fibonacci(sf)
73 print(estadisticas(sf))
```

Parte 3 (p3.py; 2 puntos)

Ahora crea una copia de tu programa de la parte anterior (p2.py) y llámala **p3.py**. Elimina los códigos de prueba en p3.py. Deja solamente las funciones implementadas. Luego haz lo siguiente:

1. [.5] Agrega a tu programa un menú de texto que ofrezca al usuario las siguientes opciones:

1. Crear matriz Fibonacci
2. Mostrar matriz
3. Desordenar matriz
4. Extraer parte de matriz Fibonacci
5. Estadísticas de matriz Fibonacci

6. Salir

2. [1.5] Haz que cada una de las opciones del menú invoque a las funciones creadas anteriormente en este examen.

- [3] La primera opción para crear matriz debe pedir la cantidad de filas y de columnas, y llamar a `fibonacci(n, filas, columnas)` para generar una nueva matriz. El usuario debe ingresar un nombre para la nueva matriz, y dicha matriz debe quedar guardada en un diccionario con su nombre (similar a cómo en la tarea se mantienen las imágenes disponibles).
- [3] La segunda opción debe preguntar al usuario el nombre de la matriz a desplegar y luego llamar a la función `despliega_fibonacci(l)`, pasando dicha matriz obtenida desde el diccionario.
- [3] La tercera opción debe preguntar al usuario el nombre de la matriz a desordenar, crear una copia de dicha matriz (se puede llamar a `sub_fibonacci(l, fila, columna)`, pasando valor 1 a `fila` y `columna`), luego llamar a la función `desordenar(l)` pasando la copia de la matriz, y finalmente, pedir al usuario un nombre con el cual guardar en el diccionario la matriz desordenada.
- [3] La cuarta opción también debe preguntar al usuario el nombre de la matriz a la cual se le extraerá una parte, la fila y la columna a partir de donde extraer, y luego llamar a la función `sub_fibonacci(l, fila, columna)`. La función luego debe pedir al usuario un nombre para guardar la parte (submatriz) extraída en el diccionario.
- [3] La quinta opción debe pedir al usuario el nombre de una matriz y luego llamar a `estadísticas(l)`, pasando la matriz `l`. Con los resultados obtenidos, mostrar el promedio, el valor mínimo y el valor máximo de la matriz.

Recuerda revisar el vídeo de la parte 3 en <https://bit.ly/20240517-ing1103-m3> para ver la salida esperada del programa.

Solución

Listing 5: Solución de la Parte 3

```
1  from numpy import random
2
3  def fibonacci(n, filas, columnas):
4      l = [0,1]
5      m = []
6      count = 0
7      for i in range(filas):
8          if i == 0:
9              fila = [0,1]
10             count += 2
11         else:
12             fila = []
13             for j in range(columnas-len(fila)):
14                 l.append(l[-2:-1][0] + l[-1:-][0])
15                 if count < n:
16                     fila.append(l[-1])
17                     count += 1
18             else:
19                 fila.append(0)
20             m.append(fila)
21         return m
22
23 def despliega_fibonacci(l):
24     for i in range(len(l)):
25         for j in range(len(l[i])):
26             print(f"{l[i][j]:<9} ", end="")
27         print()
```

```
28
29 def desordenar(l):
30     for i in range(len(l)):
31         for j in range(len(l[0])):
32             ir = random.randint(len(l))
33             jr = random.randint(len(l[0]))
34             aux = l[i][j]
35             l[i][j] = l[ir][jr]
36             l[ir][jr] = aux
37
38 def estadisticas(l):
39     suma = 0
40     n = len(l) * len(l[0])
41
42     mn = l[0][0]
43     mx = l[0][0]
44
45     for i in range(len(l)):
46         for j in range(len(l[0])):
47             suma += l[i][j]
48             if l[i][j] < mn:
49                 mn = l[i][j]
50             if l[i][j] > mx:
51                 mx = l[i][j]
52     return [suma/n, mn, mx]
53
54 def sub_fibonacci(l, fila, columna):
55     mfb = []
56     for i in range(fila-1, len(l)):
57         mfb.append(l[i][columna-1:])
58     return mfb
59
60 def menu():
61     print("1. Crear matriz Fibonacci\n",
62           "2. Mostrar matriz\n",
63           "3. Desordenar matriz\n",
64           "4. Extraer parte de matriz Fibonacci\n",
65           "5. Estadísticas de matriz Fibonacci\n",
66           "6. Salir\n", sep="")
67     opcion = input(">")
68     return opcion
69
70 def preguntar_matriz(matrices):
71     print("¿Cuál matriz usar?")
72     print(" ".join(matrices.keys()))
73     nombre = input("nombre>")
74     return nombre
75
76 matrices = {}
77
78 opcion = menu()
79
80 while opcion != '6':
81     if opcion == '1':
82         n = int(input("n>"))
83         filas = int(input("filas>"))
84         columnas = int(input("columnas>"))
85         nombre = input("nombre>")
86         matrices[nombre] = fibonacci(n, filas, columnas)
87         despliega_fibonacci(matrices[nombre])
88     if opcion == '2':
89         nombre = preguntar_matriz(matrices)
90         despliega_fibonacci(matrices[nombre])
91     if opcion == '3':
92         nombre = preguntar_matriz(matrices)
93         cpm = sub_fibonacci(matrices[nombre], 1, 1)
94         desordenar(cpm)
95         nombre_nva = input("¿Con qué nombre guardar matriz desordenada?")
96         matrices[nombre_nva] = cpm
97     if opcion == '4':
98         nombre = preguntar_matriz(matrices)
99         fila_ini = int(input("fila inicial>"))
100        col_ini = int(input("columna inicial>"))
```

```
101     sbm = sub_fibonacci(matrices[nombre], fila_ini, col_ini)
102     nombre_nva = input("¿Con qué nombre guardar matriz?")
103     matrices[nombre_nva] = sbm
104 if opcion == '5':
105     nombre = preguntar_matriz(matrices)
106     stats = estadisticas(matrices[nombre])
107     print(f"Promedio: {stats[0]}, minimo: {stats[1]}, maximo: {stats[2]}")
108 opcion = menu()
```
